SRD: A Multi-Tenant Architecture for Digital Management of Community Action Boards

Juan Sebastián González Horta
Servicio Nacional de Aprendizaje (SENA) · Neiva, Colombia

Abstract— Community Action Boards require secure, organization-scoped tools for people records, activities, financial entries and communications. This applied technological case study examines the SRD multi-tenant platform through 19 October 2026. Evidence consists of code and migration review, domain tests, operational logs, and directed browser runs on synthetic data. The implemented prototype contains an API Gateway, ten Laravel domain services, per-domain MySQL databases and a React web client; Windows and Android clients are partial. On one Docker-served web image, 33 distinct browser test files passed among 140 present; the other 107 lack a recorded final run on that image. The result is not a pass rate for the entire suite or a measure of system completion. Expired antivirus signatures blocked upload checks, while role-based writes, full regression, user evaluation and production-grade TLS/TURN remain open. The study documents verifiable progress and its limitations, rather than certifying a deployable product.

Keywords— multi-tenancy, community organizations, microservices, auditing, security, applied research

I. INTRODUCTION

Colombian community action organizations have a distinct institutional role, addressed by Law 2166 of 2021 [7]. Their daily administration requires maintaining membership records, scheduling activities, storing documents, tracking resources, and identifying who changed information. When these activities depend on disconnected files and applications, administrators face difficulties reconstructing versions, enforcing permissions, and maintaining continuity. SRD approaches this operational challenge as a service-oriented software project; it does not replace the legal framework governing community organizations.

The research question guiding this increment was: how can a shared digital platform be built for multiple Community Action Boards while preserving organizational data separation and auditable changes through independently managed modules? The design assumption was that an API Gateway resolving authenticated sessions, compulsory organization context in each domain, and deferred publication of audit events can support these constraints as the platform grows.

The general objective was to design and technically validate an increment of SRD for the administration of multiple community boards. Specific objectives were to define the problem and business rules; organize clients, services and databases into explicit boundaries; implement representative web flows and partial companion clients; and collect test evidence and incident records so that implementation progress would not be confused with final acceptance.

II. RELATED WORK AND TECHNOLOGY SURVEILLANCE

Multi-tenant architecture permits multiple organizations to use shared application infrastructure, but introduces explicit trade-offs in data partitioning, customization and recovery. Tsai et al. [1] discuss such SaaS concerns; Krebs, Momm and Kounev [2] distinguish tenancy from the mere sharing of computing resources. This distinction matters for SRD: its MySQL databases are partitioned by functional domain, not by individual community board. Organization-level context separates records and operations within each domain.

Security does not follow automatically from placing an access check at the entry point. Pervez et al. [3] explore secure multi-tenant SaaS architecture; Singh et al. [4] address protections among tenants. Krebs and Loesch [5] show that performance isolation and data isolation require different reasoning, while Reinhardt et al. [6] discuss provisioning choices in mixed-tenancy environments. Collectively, these contributions motivate evaluating authorization, data ownership, operational recovery, and concurrent performance as distinct engineering questions.

The technical review also used authoritative guidance. Colombian Law 2166 [7] establishes the relevant organizational setting; OWASP ASVS 5.0 [8] provides a security verification framework for web applications; and NIST SP 800-63B-4 [9] addresses authentication and authenticator management. SRD draws on these resources when defining testable controls, including backend authorization, session handling and an additional verification step. The project is not claimed to be certified against these standards.

Comparable products include CiviCRM [10], an open-source constituent-management system for non-profits, and enterprise platforms such as Odoo [11] and ERPNext [12], which support inventory and operational workflows. These examples demonstrate existing solution patterns, not a feature-by-feature equivalence. The targeted gap for SRD is narrower: the coordination of people, events, internal records and informative treasury tasks under community-board-specific roles. No claim of superior breadth or commercial performance is made.

III. METHODOLOGY

The observation window was extended to 19 October 2026 to include the directed browser runs reported in REVISION_UX_WEB.md and PENDIENTES-WEB-2026-10-16.md, cross-checked against VALIDACION.md and

TRAZABILIDAD.md. The counting unit is a distinct browser test file reported as passing on one Docker-served web image; different files contain different numbers of test cases. Accordingly, these observations cannot be treated as a product-completion measure or a whole-suite pass rate.

This project used an applied, descriptive technology case-study approach. The unit of analysis was the SRD 0.1 working increment available on October 9, 2026. Primary materials included the requirements specification, source code, database migrations, OpenAPI contracts, Docker Compose configuration, automated test reports, and operational recovery logs. Literature was examined around multi-tenancy, security, digital identity, and comparable platforms.

The work followed five phases. First, a baseline comprising 56 functional requirements, 18 non-functional requirements and 20 use cases was established for community-board activities. Second, service boundaries and a gateway were selected. Third, representative local flows were implemented and exercised with synthetic test data. Fourth, records from unit, browser, and operational checks were classified by evidence type. Fifth, corrective diagnosis was carried out on migration and audit-delivery failures in a Docker development environment.

The instruments comprised requirements traceability, functional test cases, PHP/Jest/Playwright results, Docker health checks, and transactional outbox counters. No end-user interviews, questionnaires, or satisfaction measurements are reported here because no verifiable primary user dataset was available. That work belongs to the next validation phase rather than being inferred from engineering test reports.

To avoid overstatement, each claim was classified as an automated test documented in the repository, a manual operational observation, or an outstanding feature. Healthy containers were not treated as proof that every business requirement was accepted. Likewise, the project did not present an integrated concurrency benchmark for the complete platform.

A practical consistency matrix linked each research objective to a means of verification. Organization-level separation was mapped to context validation in the Gateway and domains; traceability was mapped to event contracts and outbox records; and maintainability was mapped to migration, test and backup scripts. This mapping ensures that a technical statement can be traced back to a file and a reproducible check. It does not, however, substitute for an independent review of the complete system.

Case selection emphasized higher-risk flows: membership authentication, organization-scoped queries, concurrent folder changes, informative treasury movements, inventory histories, and audit delivery. Demonstration fixtures were deliberately synthetic to avoid using real residents, private email messages, or financial transactions during local experiments. The review retained only test outcomes explicitly supported by repository validation notes or by operational logs generated during the documented recovery session.

The literature search used terms including multi-tenancy AND security, multi-tenant SaaS AND architecture, community organization information systems, and equivalent Spanish phrases. Papers [1]–[6] were selected for their treatment of resource isolation, protection and architecture; references [7]–[9] for legal and technical guidance; and [10]–[12] as examples of existing solutions. This is a focused exploratory review, not an exhaustive systematic review or an empirical feature benchmark across products.

IV. SYSTEM DESIGN AND IMPLEMENTATION

Subsequent interface work addressed empty, loading and failure states at narrow and desktop widths, along with explicit retries and navigation/permission checks across six roles. In the People lookup, the previous record is now cleared when a different query starts or fails with HTTP 503. The Windows and Android distribution paths, a specialized Reports service and a public-grade TLS/TURN configuration remain outside the accepted implementation.

The web interface is built with React 19, TypeScript and Vite. Laravel powers the entry Gateway and domain applications; MySQL 8.4 provides per-domain databases. Ten domains are present: identity, configuration, records, audit, files, calendar, notifications, treasury, inventory and chat. A separate specialized reporting service remains within the planned architecture, not the implemented code. The Electron Windows client uses the web application hosted on the server, while the native Android client and an Expo Go variant cover only selected workflows.

The Gateway validates a session and forwards a signed internal context containing user, organization, and permissions. Each domain revalidates access to its own resources, so hiding interface controls does not substitute for server authorization. Databases and migrations belong to their respective domains. Although several boards share service infrastructure, record lookups must remain restricted to the verified active organization and cannot rely on a tenant identifier freely supplied by a client.

Auditing uses a transactional outbox. The originating service stores an event together with its business change, and a scheduler attempts delivery to the audit service. Stable event identifiers allow the receiver to avoid duplicate stored audit rows after retries. Eventual delivery introduces a visible operational state: an event may be ready, deferred, delivered or exhausted. SRD exposes counters to help administrators identify these conditions rather than assuming every event reaches its target immediately.

The Files domain uses private storage, upload constraints and ClamAV scanning for applicable content. Treasury maintains informative COP entries and corrective reversals rather than editing confirmed records. Inventory maintains asset histories through movements. Such decisions deliberately place consistency and traceability mechanisms in backend rules and database operations, not merely in forms displayed by a browser.

V. RESULTS

As of 19 October 2026, 33 distinct browser test files had passed among 140 available files on the same Docker-served web image. The remaining 107 lack a recorded final run on that image; no whole-suite pass rate can be inferred. web-empty-states.spec.mjs covered 14 empty-state sections at 320 and 1280 pixels (28 screenshots). Directed checks also covered slow reads and retry after a 503 error in People lookup. Three suites—service-status.spec.mjs, person-date-ranges.spec.mjs and person-base-filters.spec.mjs—passed individually after outdated test expectations were revised.

An extensive browser batch made Docker Desktop unresponsive. A controlled restart restored an HTTP 200 web response and reported 23 healthy services at that moment, but did not finish the pending regression. Upload checks were blocked by expired ClamAV signatures: HTTP 503 reflected a safe rejection, not successful scan coverage.

The examined repository contains the API Gateway, ten operational domain implementations, a React web application and partial desktop and Android clients. Its requirements traceability retains 56 functional requirements, 18 non-functional requirements and 20 use cases, whose individual acceptance status remains open; development categories must not be converted into a completion percentage. The recorded Files unit tests passed 32 cases with 778 assertions. Separate Chromium runs passed a bilingual folder test and a hierarchy/conflict regression. Android development evidence includes 37 successful Jest tests and a built x86_64 debug package.

On October 9, the audit service database was initially missing its audit-event table, which caused sender outboxes to exhaust delivery attempts. After applying the missing migration and backing up affected databases, 15 Identity events, eight Records events, four Calendar events, three Treasury events, and three Inventory events were recovered. All 33 were subsequently reported as published with zero exhausted in those emitters. Files and Chat migrations were applied, and their scheduler containers eventually transitioned to healthy.

A second incident concerned the signature updater. ClamAV itself was healthy, but the FreshClam script failed because shell input contained Windows CRLF line endings. Converting the script to LF and recreating only the updater enabled the signature database test and upgraded daily.cld to version 28148. The marker file was visible from the Files service. This is evidence of signature update and availability, not a completed end-to-end malicious-file rejection test.

The demonstration fixture contains eight pending persons, four calendar events, three informative treasury entries and three inventory assets, all synthetic. A previous physical Android Expo Go run exercised login, email MFA, Home, People, Agenda, Alerts, Profile and session persistence using an earlier, smaller fixture. No comprehensive user-experience metric or production security audit has been established.

Regarding authorization, technical materials show that the Gateway resolves the session and signs the context sent to each domain. This leads to a testable implication: a hidden button is not an access-control mechanism in itself; backend tests must verify operations for each role. The examined project includes a role matrix covering SA, AD, RE, TE, AU and CO as well as delegated folder-reading permissions. Such rules require regression testing after membership changes, session revocations, and organization suspension.

Regarding persistence, domain-specific OpenAPI contracts and migrations are present in the repository. The Files and Chat incident illustrates why those two artifacts must be checked together: an HTTP process could be available while the corresponding database still had no tables. After applying migrations, outbox queries no longer reported the missing table and the two scheduler containers reached healthy status. Although narrowly scoped, this observation provides reproducible evidence that an infrastructure availability signal does not prove functional readiness.

The findings should be interpreted at three separate levels: infrastructure deployment, component-level test results, and acceptance of actual community-board administrative workflows. Docker health checks and migrations address the first; automated suites and browser flows address the second; the third remains open. Maintaining these distinctions makes it possible to report real engineering progress without inventing completion percentages for the 56 requirements or extrapolating local evidence to a public deployment.

VI. DISCUSSION AND LIMITATIONS

The evidence is bounded by local synthetic fixtures, selected browser runs and no formal end-user evaluation. Full visual review, the allowed/denied write matrix for all six roles and a complete 140-file regression remain open. ClamAV signatures must be refreshed before upload acceptance, while production recovery, concurrent load, HTTPS/TURN between networks and distributed installers still require independent testing. Container health, browser files and actual acceptance outcomes are distinct observations.

These results illustrate both the benefit and operational cost of modular design. The missing audit table affected event delivery but could be diagnosed without deleting

business records. The signature updater was repaired without reinstalling the database. At the same time, each independent service carries separate migration, configuration, health-check, and integration responsibilities. Creating architectural boundaries therefore improves fault localization but does not remove the work of operating them.

The outbox provides stronger traceability under transient failures, although exhausted events still require an authorized recovery procedure and monitoring. A healthy malware scanner does not replace file-type controls, quotas or access checks. Similarly, maintaining many organizations in one application requires repeatable cross-tenant authorization tests; it would be unwarranted to infer complete isolation from a single successful demonstration.

Important limitations remain: there is no documented pilot with real community boards; functional acceptance has not been completed; a specialized reports service is missing; and chat, Android and Windows functions are incomplete. The platform has not been benchmarked for fifty simultaneous users. Production-grade HTTPS, operational continuity, and signed installer distribution have not been demonstrated. Consequently, the results establish a verifiable development increment rather than an accepted final product.

VII. CONCLUSIONS AND FUTURE WORK

Future work is measurable: run and archive results for all 140 browser files against one final image, restore valid antivirus definitions and retest uploads, complete role-based write and accessibility checks, rehearse recovery with privacy suppressions, benchmark concurrent use, and seek authorized user feedback. These are proposed next steps rather than outcomes of the present study.

SRD demonstrates a technically verifiable foundation for a shared digital platform for community organizations, using explicit organization context, per-domain databases and observable audit delivery. Automated checks, Docker diagnostics and narrowly scoped corrective actions made it possible to validate specific engineering mechanisms while retaining a realistic record of limitations.

The next stage should complete outstanding requirements, execute systematic cross-organization access tests, validate clean-file and EICAR scanning, measure concurrent load, apply instruments to actual users and prepare a secure production deployment. The central methodological lesson is that developing a feature and demonstrating that it satisfies an acceptance criterion are different activities. Reliable claims require identifiable tests, synthetic or consented datasets, dates and reproducible evidence.

REFERENCES

[1] W.-T. Tsai, Q. Shao, Y. Huang, and X. Bai, “Towards a scalable and robust multi-tenancy SaaS,” 2010. doi:10.1145/2020723.2020731.

[2] R. Krebs, C. Momm, and S. Kounev, “Architectural Concerns in Multi-Tenant SaaS Applications,” 2012. https://www.scitepress.org/PublishedPapers/2012/39576/pdf/index.html

[3] Z. Pervez, S. Lee, and Y.-K. Lee, “Multi-Tenant, Secure, Load Disseminated SaaS Architecture,” ICACT, 2010. https://www.icact.org/upload/2010/0265/20100265_finalpaper.pdf

[4] A. Singh, S. Kumar J., and A. Muralidhar, “A Secure Multi-Tenant Model for SaaS System,” International Journal of Computer Applications, vol. 90, no. 15, 2014. doi:10.5120/15793-4468.

[5] R. Krebs and M. Loesch, “Comparison of Request Admission Based Performance Isolation Approaches in Multi-tenant SaaS Applications,” 2014. https://www.scitepress.org/PublishedPapers/2014/49488/pdf/index.html

[6] M. Reinhardt, S. T. Ruehl, S. A. W. Verclas, U. Andelfinger, and A. Schütte, “Architectural Design of a Deployment Platform to Provision Mixed-tenancy SaaS-Applications,” 2014. https://www.scitepress.org/PublishedPapers/2014/48138/pdf/index.html

[7] Congress of Colombia, “Law 2166 of 2021: community action organizations,” 2021. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=184758

[8] OWASP Foundation, “Application Security Verification Standard (ASVS),” version 5.0.0, 2025. https://github.com/OWASP/ASVS

[9] NIST, “Digital Identity Guidelines: Authentication and Authenticator Management,” SP 800-63B-4, 2025. doi:10.6028/NIST.SP.800-63B-4.

[10] CiviCRM, “CiviCRM Core: constituent relationship management for non-profit organizations,” https://github.com/civicrm/civicrm-core

[11] Odoo, “Inventory: Features,” https://www.odoo.com/app/inventory-features

[12] Frappe Technologies, “ERPNext,” https://erpnext.com

[13] J. S. González Horta, “SRD: README, VALIDACION, TRAZABILIDAD, ARQUITECTURA and source code,” project repository, working version dated October 9, 2026.

TOOLS DISCLOSURE

Digital assistance was used for drafting and language review. The author checked technical statements against source code, test logs, and cited materials; no fictitious survey data or experimental outcomes were generated.